SolarHub PostgreSQL Production Setup

Single-server Ubuntu/Debian deployment with Docker and Kamal

Prepared October 20, 2026

This guide installs PostgreSQL directly on the same server that runs the SolarHub Docker containers. It creates a restricted PostgreSQL role and four production databases, then permits database connections only from Kamal's Docker network.

Important security rule

Do not expose PostgreSQL port 5432 to the public internet. The database should listen only on localhost and the private Docker network gateway.

1. Install PostgreSQL

Run these commands on the server from an administrator account with sudo access:

```
sudo apt update
sudo apt install -y postgresql postgresql-client
sudo systemctl enable --now postgresql
sudo systemctl status postgresql --no-pager
psql --version
```

The status command should report that PostgreSQL is active. Record the major version printed by psql; it is used in configuration paths later.

2. Create the Kamal Docker network

Run this as the deploy user:

```
docker network inspect kamal >/dev/null 2>&1 || docker network create kamal
```

Display the network subnet and host gateway:

```
docker network inspect kamal \
  --format 'Subnet={{(index .IPAM.Config 0).Subnet}} Gateway={{(index .IPAM.Config 0).Gateway}}'
```

Record both values. A typical result looks like this:

```
Subnet=172.18.0.0/16 Gateway=172.18.0.1
```

The rest of this guide calls these values DOCKER_SUBNET and DOCKER_GATEWAY. Use the actual values reported by your server.

3. Configure PostgreSQL network access

List the installed PostgreSQL version directory:

```
ls /etc/postgresql
```

Edit the server configuration, replacing VERSION with that directory name:

```
sudoedit /etc/postgresql/VERSION/main/postgresql.conf
```

Set these values, replacing DOCKER_GATEWAY with the network gateway recorded above:

```
listen_addresses = 'localhost,DOCKER_GATEWAY'
password_encryption = 'scram-sha-256'
```

Example:

```
listen_addresses = 'localhost,172.18.0.1'
password_encryption = 'scram-sha-256'
```

Do not use an asterisk for listen_addresses. Restart PostgreSQL:

```
sudo systemctl restart postgresql
```

4. Create the restricted SolarHub role

Generate a long random password and save it in a password manager. Then create the PostgreSQL login role:

```
sudo -u postgres createuser --login --pwprompt --no-superuser --no-createdb --no-createrole solarhub
```

Enter the generated password when prompted. This role deliberately cannot create databases, roles, or superusers.

5. Create the production databases

Because the SolarHub role does not have CREATEDB permission, create the four production databases once using the PostgreSQL administrator:

```
sudo -u postgres createdb --owner=solarhub solarhub_production
sudo -u postgres createdb --owner=solarhub solarhub_production_cache
sudo -u postgres createdb --owner=solarhub solarhub_production_queue
sudo -u postgres createdb --owner=solarhub solarhub_production_cable
```

Rails will not need permission to create databases. During deployment, bin/rails db:prepare will create tables and run migrations inside these existing databases.

Verify the role and databases:

```
sudo -u postgres psql -c "\du solarhub"
sudo -u postgres psql -l
```

The SolarHub role should own all four databases and should not be a superuser.

6. Authorize connections from Docker

Edit the client-authentication configuration:

```
sudoedit /etc/postgresql/VERSION/main/pg_hba.conf
```

Add these lines at the end of the file. Replace DOCKER_SUBNET with the actual network subnet:

```
host  solarhub_production        solarhub  DOCKER_SUBNET  scram-sha-256
host  solarhub_production_cache  solarhub  DOCKER_SUBNET  scram-sha-256
host  solarhub_production_queue  solarhub  DOCKER_SUBNET  scram-sha-256
host  solarhub_production_cable  solarhub  DOCKER_SUBNET  scram-sha-256
```

Example:

```
host  solarhub_production        solarhub  172.18.0.0/16  scram-sha-256
host  solarhub_production_cache  solarhub  172.18.0.0/16  scram-sha-256
host  solarhub_production_queue  solarhub  172.18.0.0/16  scram-sha-256
host  solarhub_production_cable  solarhub  172.18.0.0/16  scram-sha-256
```

Check for pg_hba.conf parsing errors:

```
sudo -u postgres psql -c "SELECT line_number, error FROM pg_hba_file_rules WHERE error IS NOT NULL;"
```

The query should return zero rows. Restart PostgreSQL:

```
sudo systemctl restart postgresql
```

Confirm that PostgreSQL listens only on localhost and the Docker gateway:

```
sudo ss -ltnp | grep 5432
```

The output should not contain 0.0.0.0:5432 or the server's public IP address.

7. Configure the firewall

Check whether UFW is active:

```
sudo ufw status
```

If it is active, allow only the Docker subnet to reach PostgreSQL on the Docker gateway:

```
sudo ufw allow from DOCKER_SUBNET to DOCKER_GATEWAY port 5432 proto tcp
```

Example:

```
sudo ufw allow from 172.18.0.0/16 to 172.18.0.1 port 5432 proto tcp
```

Do not add a public allow rule for port 5432.

8. Test from a Docker container

Replace POSTGRES_MAJOR with the PostgreSQL major version and DOCKER_GATEWAY with the actual gateway. Docker may download the matching client image the first time this command runs:

```
docker run --rm -it --network kamal postgres:POSTGRES_MAJOR-alpine \
  psql --host=DOCKER_GATEWAY --username=solarhub \
  --dbname=solarhub_production --password \
  --command='select current_database(), current_user;'
```

Enter the SolarHub database password when prompted. Successful output identifies solarhub_production as the current database and solarhub as the current user. Do not continue to deployment until this test passes.

9. Configure Kamal

In config/deploy.yml, add the PostgreSQL settings. Replace DOCKER_GATEWAY with the actual gateway:

```
env:
  secret:
    - RAILS_MASTER_KEY
    - POSTGRES_PASSWORD

  clear:
    SOLID_QUEUE_IN_PUMA: true
    POSTGRES_HOST: DOCKER_GATEWAY
    POSTGRES_USER: solarhub
    POSTGRES_DB: solarhub_production
    POSTGRES_CACHE_DB: solarhub_production_cache
    POSTGRES_QUEUE_DB: solarhub_production_queue
    POSTGRES_CABLE_DB: solarhub_production_cable
```

In .kamal/secrets, add a reference to the environment variable. Do not put the actual password in this

tracked file:

```
POSTGRES_PASSWORD=$POSTGRES_PASSWORD
```

Inside the development container, load the password without placing it in shell history:

```
read -rsp "PostgreSQL password: " POSTGRES_PASSWORD
export POSTGRES_PASSWORD
echo
```

Check the resulting deployment configuration:

```
bin/kamal config
```

10. Completion checklist

- PostgreSQL is active and enabled at boot.
- The solarhub role is not a superuser and cannot create databases or roles.
- All four production databases exist and are owned by solarhub.
- PostgreSQL listens only on localhost and the Kamal Docker gateway.
- pg_hba.conf permits SCRAM authentication only from the Kamal subnet.
- The Docker-based connection test succeeds.
- The PostgreSQL password is stored in a password manager and is not committed.
- Kamal receives POSTGRES_PASSWORD as a secret.

11. Official references

Ubuntu PostgreSQL installation and configuration:

```
https://documentation.ubuntu.com/server/how-to/databases/install-postgresql/
```

PostgreSQL role documentation:

```
https://www.postgresql.org/docs/current/sql-createrole.html
```

PostgreSQL client authentication documentation:

```
https://www.postgresql.org/docs/current/auth-pg-hba-conf.html
```

Kamal Docker network documentation:

```
https://kamal-deploy.org/docs/upgrading/network-changes/
```